Le langage Python

Structures de données

Dominique FRANCISCI

(2020-2026)



Objectif de ce support

Etre capable d'utiliser les structures de données Python :

- Chaînes de caractères `'This is a string !'`
- Tuples
- Listes
- Dictionnaires
- Ensembles



Sommaire



- **Données simples/composites**
- Chaînes de caractères
- Tuples
- Listes
- Dictionnaires
- Ensembles
- Instruction for
- Comparaison for / while
- Listes complexes

Données simples / composites

- Les types int, float et bool sont appelés des types simples
- Leurs valeurs, appelées constantes littérales, sont simples
- Il existe également des données composées de plusieurs valeurs : les données composites
- Les cinq structures de données suivantes sont abordées :
  - Les chaînes de caractères
  - Les tuples
  - Les listes
  - Les dictionnaires
  - Les ensembles

Où en sommes-nous ?

- Données simples/composites
- **Chaînes de caractères**
- Tuples
- Listes
- Dictionnaires
- Ensembles
- Instruction for
- Comparaison for / while
- Listes complexes



Chaînes de caractères

- Une chaîne de caractères est une séquence de caractères unicode
- C'est une suite d'éléments ordonnés, indicés par des entiers
- L'opérateur [i] avec i entier (positif ou négatif) permet d'accéder aux différents caractères de la chaîne

Exemple



```
s = 'Qui veut la paix prépare la guerre.'
print(s[0], s[6], s[23], s[32])

>>>
  Q u e r
s = 'Si vis pacem, para bellum.'
print(s[-1], s[-3], s[-12], s[-14])

>>>
. u p ,
```

Chaînes de caractères

Parcours d'une chaîne à l'endroit et à l'envers

Exemple : à l'endroit



```
s = 'Python'
i = 0
while i < len(s):
    print(s[i], end=' ')
    i = i + 1
>>>
  P y t h o n
```

Exemple : à l'envers



```
s = 'Python'
i = -1
while -len(s) <= i:
    print(s[i], end=' ')
    i = i - 1
>>>
  n o h t y P
```

Extraction de sous-chaîne

- Syntaxe : [b:e]

  avec : « b » indice de début et « e » indice de fin
- Représente les caractères entre les indices b et e (exclus)

Exemple 1



```
>>> s = 'Cogito ergo sum'
>>> s[7:11]
  'ergo'
>>> s[:6]
  'Cogito'
>>> s[7:]
  'ergo sum'
```

Exemple 2



```
>>> s = 'Cogito ergo sum'
>>> s[-8:-4]
  'ergo'
>>> s[:-9]
  'Cogito'
>>> s[-8:]
  'ergo sum'
```

Extraction de sous-chaîne

- Syntaxe : [b:e:k]

  avec : « b » indice de début, « e » indice de fin et « k » un pas :
- Représente les caractères entre les indices b et e (exclus) avec un pas k :

  i, i+k, i+2k, i+3k, …

Exemple 1

```
>>> s = 'Cogito ergo sum'
>>> s[0:6:2]
  'Cgt'
>>> s[:11:3]
  'Ci g'
>>> s[10:6:-1]
  'ogre'
```

Exemple 2

```
>>> s = 'Cogito ergo sum'
>>> s[-15:-9:2]
  'Cgt'
>>> s[:-4:3]
  'Ci g'
>>> s[-5:-9:-1]
  'ogre'
```

Opérations sur les chaînes de caractères

Opérateurs

- **+** : concaténation
- ***** : répétition
- **len** : fonction retournant la longueur d'une chaîne

Exemple



```
>>> s1, s2 = 'Introduction', ' au langage Python !'
>>> s1[0:5] + s2[0::3]
  'Intro  ngPh'
>>> len(s1[0:5] + s2[11:])
   14
```

Chaînes de caractères

Exercice (1/3)



Ecrire un script qui détermine et qui affiche le nombre d'occurrences de la lettre « a » (minuscule, majuscule ou avec un accent grave) dans une chaîne de caractères saisie au clavier par l'utilisateur.



Chaînes de caractères

Correction exercice 1/3

```
a_count = 0
i = 0
s = input('Veuillez saisir une chaîne de caractères : ')

while i < len(s) :
  if s[i] == 'a' or s[i] == 'à' or s[i] == 'A' :
    a_count = a_count + 1
  i = i+1

print('Total :', a_count)
```



Chaînes de caractères

Exercice (2/3)



Ecrire un script qui, à partir d'une chaîne de caractères saisie par l'utilisateur, créé une nouvelle chaîne en remplaçant :

- *tous les « a » par des « 1 »*
- *tous les « b » par des « 2 »*
- *et tous les « c » par des « 3 »*



Chaînes de caractères

Correction exercice 2/3

```
s = input('Veuillez saisir une chaîne de caractères : ')
new_s = ''
i = 0
while i < len(s) :
  if s[i] == 'a' :
    new_s = new_s + '1'
  elif s[i] == 'b' :
    new_s = new_s + '2'
  elif s[i] == 'c' :
    new_s = new_s + '3'
  else :
    new_s = new_s + s[i]
  i = i+1
print('Nouvelle chaîne :', new_s)
```



Chaînes de caractères

Exercice (3/3)



Ecrire un script qui concatène une série de chaînes de caractères en les séparant par des espaces ; la saisie s'arrête quand la chaîne saisie est vide.



Chaînes de caractères

Correction exercice 3/3



```
words = ''
continuation = True

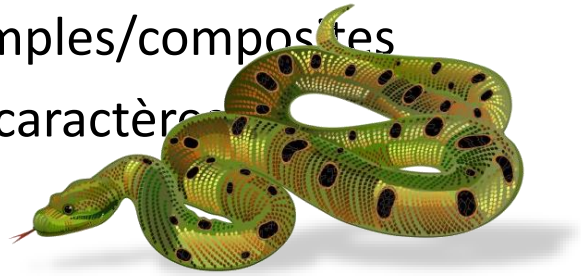
while continuation :
  s = input('Veuillez saisir une chaîne de caractères : ')
  words = words + ' ' + s
  continuation = (s != '')

print('Résultat :', words)
```



Où en sommes-nous ?

- Données simples/composites
- Chaînes de caractères
- **Tuples**
- Listes
- Dictionnaires
- Ensembles
- Instruction for
- Comparaison for / while
- Listes complexes



Tuples

Le type tuple

- Le type tuple est une séquence d'éléments hétérogènes non modifiables

- Syntaxe : (element_1, element_2, …, element_n)
  - Les éléments ne sont pas forcément du même type (hétérogènes)
  - Tuple vide : (,)
  - Tupe à un seul élément : (element,)
  - Les éléments sont indicés de 0 à n-1 (indices récupérables par la fonction enumerate)
  - Les éléments d'un tuple ne peuvent pas être modifiés

Tuples

Exemple



```
>>> t = (1, 'deux', 3.0, 4, 'cinq')
>>> t[0]
  1
>>> t[2]
  3.0
>>> t[1:4]
  ('deux', 3.0, 4)
>>> t[0] = 0
  TypeError          Traceback (most recent call last)
  <ipython-input-5-9946ebc62b5f> in <module>
  ----> 1 t[0]=0
```

Initialisation des tuples

La fonction range

- **range** retourne un séquence d'entiers pouvant servir à :
  - L'initialisation de listes
  - L'initialisation de tuples
  - Au parcours d'objets composites indexés par des entiers

- **range**(e) correspond à la suite 0, 1, …, e-1

- **range**(b, e) correspond à la suite b, b+1, …, e-1

- **range**(b, e, s) correspond à la suite b, b+s, b+2s, b+3s, …, e-1

Initialisation des tuples

Exemple



```
>>> t = tuple(range(4))
  (0, 1, 2, 3)

>>> t = tuple(range(2, 5))
  (2, 3, 4)

>>> t = tuple(range(3, 15, 4))
  (3, 7, 11)
```

Opérations sur les tuples

Opérateurs

- + : concaténation
- * : répétition
- **len** : fonction retournant la longueur d'un tuple

Exemple



```
>>> t1, t2 = (1, 2, 3, 4), (-1, -2, -3, -4)
>>> t1[:3] + t2[2:]
  (1, 2, 3, -3, -4)
>>> t1[-4:-1]*2
  (1, 2, 3, 1, 2, 3)
>>> len(t1 + t2[::2])
   6
```

Où en sommes-nous ?

- Données simples/composites
- Chaînes de caractères
- Tuples
- **Listes**
- Dictionnaires
- Ensembles
- Instruction for
- Comparaison for / while
- Listes complexes



Listes

Le type list

- Le type list est une séquence d'éléments hétérogènes modifiables (contrairement au tuple)

- Syntaxe : [element_1, element_2, …, element_n]
  - Les éléments ne sont pas forcément du même type (hétérogènes)
  - Liste vide : []
  - Liste à un seul élément : [element]
  - Les éléments sont indicés de 0 à n-1 (indices récupérables par la fonction enumerate)
  - Les éléments d'une liste peuvent être modifiés

Listes

Exemple (1/2)



```
>>> l = [1, 'deux', 3.0, 4, 'cinq']
>>> l[0]
  1
>>> l[2]
  3.0
>>> l[1:4]
  ['deux', 3.0, 4]
>>> l[0] = 0
>>> l
  [0, 'deux', 3.0, 4, 'cinq']
```

Listes

Exemple (2/2)



```
>>> l1 = ['a', 'b']
>>> l2 = ['1', '2']
>>> l = [l1, l2]
  [['a', 'b'], ['1', '2']]
>>> l[0][0]
  'a'
>>> l[0][1] = 0
>>> l
  [['a', 0], ['1', '2']]
```

Initialisation des listes

La fonction range

- **range** retourne un séquence d'entiers pouvant servir à :
  - L'initialisation de listes
  - L'initialisation de tuples
  - Au parcours d'objets composites indexés par des entiers

- **range**(e) correspond à la suite 0, 1, …, e-1

- **range**(b, e) correspond à la suite b, b+1, …, e-1

- **range**(b, e, s) correspond à la suite b, b+s, b+2s, b+3s, …, e-1

Initialisation des listes

Exemple



```
>>> l = list(range(4))
  [0, 1, 2, 3]

>>> t = list(range(-4, 1))
  [-4, -3, -2]

>>> t = list(range(-4, 12, 3))
  [-4, -1, 2, 5, 8, 11]
```

Opérations sur les listes

Opérateurs

- **+** : concaténation
- ***** : répétition
- **len** : fonction retournant la longueur d'une liste

Exemple



```
>>> l1[:3] + l2[2:]
  [1, 2, 3, -3, -4]
>>> l1[-4:-1]*2
  [1, 2, 3, 1, 2, 3]
>>> len(l1 + l2[::2])
   6
```

Opérations sur les listes

Exercice (1/2)



Supposons les variables suivantes :

```
nb = '12345'
L = [nb, 3.14, ['je','tu','il','nous','vous','ils'], (-1,2)]
```

Trouver les valeurs suivantes (sans s'aider de Python) :

```
L[0]
L[1] + int(L[0][1])
(2,-1) + L[3]
L[2][1:4]
L[-2][1::2]
L[0][:2] * L[-1][-1]
```



Opérations sur les listes

Correction exercice

```
>>> nb = '12345'
>>> L = [nb, 3.14,
        ['je','tu','il','nous','vous','ils'], (-1,2)]
>>> L[0]
  '12345'
>>> L[1] + int(L[0][1])
  5.14
>>> (2,-1) + L[3]
  (2,-1,-1,2)
>>> L[2][1:4]
  ['tu','il','nous']
>>> L[-2][1::2]
  ['tu','nous','ils']
>>> L[0][:2] * L[-1][-1]
  '1212'
```



Opérations sur les listes

Exercice (2/2)



- Afficher tous les nombres pairs de 0 à 100
- Afficher tous les nombres impairs de 1 à 100
- Afficher tous les multiples de 3 jusqu'à 100
- Afficher tous les multiples de 5 jusqu'à 100
- Afficher tous les nombres de 99 à 0



Opérations sur les listes

Correction exercice

```
>>> # Nombre pairs de 0 à 100
>>> list(range(0, 102, 2))
  0 2 4 6 8 … 98 100
>>> # Nombres impairs de 1 à 100
>>> list(range(1, 100, 2))
  1 3 5 7 9 … 97 99
>>> # Multiples de 3 jusqu'à 100
>>> list(range(3, 100, 3))
  3 6 9 12 15 … 96 99
>>> # Multiples de 5 jusqu'à 100
>>> list(range(5, 105, 5))
  5 10 15 20 25 … 95 10
>>> # Nombres de 99 à 0
>>> list(range(99, -1, -1))
  99 98 97 96 95 … 1 0
```



Modification et affectation des listes

- Les listes font partie des types modifiables
- Une liste peut être modifiée :
  - par élément
  - ou par tranches d'éléments

Exemple



```
>>> l = [1, 2, 3, 4]
>>> l[0] = 0
>>> l
  [0, 2, 3, 4]
>>> l[1] = [1, 2]
>>> l
  [0, [1, 2], 3, 4]
```

```
>>> l[1:3] = [10]
>>> l
  [0, 10, 4]
>>> l[0:2] = [1, 2, 3]
>>> l
  [1, 2, 3, 4]
```

Modification et affectation des listes

- L'opérateur spécial += permet d'ajouter un/des élément(s) en fin de liste
- Syntaxe : liste += [element_1, element_2, …, element_n]

Exemple



```
>>> l = [1, 2, 3, 4]
>>> l += [5]
>>> l
  [1, 2, 3 ,4 ,5]
>>> l += [6, 7]
>>> l
  [1, 2, 3, 4, 5, 6, 7]
```

Modification et affectation des listes

- Contrairement aux type précédents, les listes :
  - ne sont pas des éléments
  - mais représentent des zones mémoires

- S'il existe plusieurs références à une même zone mémoire, alors modifier une partie de la zone à partir d'une référence modifie également les autres références

- Ce n'est pas le cas pour les autres types

Modification et affectation des listes

Exemple 1

```
>>> a = 2
>>> b = a
>>> a = 3
>>> print(a, b)
  3 2
```

Exemple 2

```
>>> l1 = [1, 2]
>>> l2 = l1
>>> l1[0] = 0
>>> print(l1, l2)
  [0, 2], [0, 2]
```

Exemple 3

```
>>> l1 = [1, 2]
>>> l2 = [1, 2]
>>> l1[0] = 0
>>> print(l1, l2)
  [0, 2], [1, 2]
```

Modification et affectation des listes

Attention aux listes de listes !

Exemple 1



```
>>> l1 = [1, 2]
>>> l2 = [1, 2]
>>> l3 = [l1, l2]
>>> print(l3)
  [[1, 2], [1, 2]]
>>> l3[0][0] = 0
>>> print(l3)
  [[0, 2], [1, 2]]
>>> l1
  [0, 2]
```

Exemple 2



```
>>> l1 = [1, 2]
>>> l2 = l1
>>> l3 = [l1, l2]
>>> print(l3)
  [[1, 2], [1, 2]]
>>> l3[0][0] = 0
>>> print(l3)
  [[0, 2], [0, 2]]
>>> l1
  [0, 2]
>>> l2
  [0, 2]
```

Modification et affectation des listes

Liste de tuples et tuple de listes

Exemple 1



```
>>> l1 = [1, 2]
>>> l2 = [3, 4]
>>> t = (l1, l2)

>>> t
  ([1, 2], [3, 4])

>>> t[0][0] = 0

>>> t
  ([0, 2], [3, 4])
```

Exemple 2



```
>>> t1 = (1, 2)
>>> t2 = (3, 4)
>>> l = [l1, l2]

>>> l
  [(1, 2), (3, 4)]

>>> l[0][0] = 0

  TypeError: 'tuple' object
  does not support item
  assignment
```

Copie de listes

Pour copier une liste il faut donc la copier élément par élément :

Script 1



```
l = [1, 2, 3, 4, 5, 6]
l_copie = []
i = 0

while i < len(l):
    l_copie += [l[i]]
    i = i+1
```

Script 2



```
l = [1, 2, 3, 4, 5, 6]
l_copie = [0]*len(l)
i = 0

while i < len(l):
    l_copie[i] += l[i]
    i = i+1
```

Copie de liste

Copie de surface d'une liste :

Exemple



```
>>> l = [[1, 2], [3, 4]]
l_copie = []
i = 0

while i < len(l) :
    l_copie += [l[i]]
    i = i+1

l[0][0] = 0
print(l, l_copie)

>>>
  [[0, 2], [3, 4]] [[0, 2], [3, 4]]
```

Copie de liste

Copie profonde d'une liste :

Exemple



```
l = [[1, 2], [3, 4]]
l_copie = []
i = 0

while i < len(l) :
    l_temp = []
    j = 0
    while j < len(l[i])
        l_temp += [l[i][j]]
        j = j+1
    l_copie += [l_temp]
    i = i+1
```

```
>>> l[0][0] = 0
>>> print(l, l_copie)

  [[0,2],[3,4]] [[1,2],[3,4]]
```

Compréhension de listes

- Il s'agit de simplifier le code pour le rendre plus lisible et donc plus rapide à écrire et plus simple à maintenir
- On parle aussi de liste en compréhension
- Syntaxe : new_list = [function(item) for item in list if condition(item)]

Exemple



```
a=[1,4,2,7,1,9,0,3,4,6,6,6,8,3]

b = []
for x in a :
  if x > 5 :
    b.append(x)

print(b)

[7, 9, 6, 6, 6, 8]
```

```
a=[1,4,2,7,1,9,0,3,4,6,6,6,8,3]

b = [x for x in a if x > 5]

print(b)

[7, 9, 6, 6, 6, 8]
```

Compréhension de listes

Exercice (1/3)



Ecrire un script qui construit une liste d'entiers à partir d'une liste de chaînes de caractères représentant des entiers.

Seuls les chaînes de 3 caractères au moins doivent être retenues.



Compréhension de listes

Correction exercice 1/3

```
# Construit une liste d'entiers à partir d'une liste de
# chaînes de caractères représentant des entiers. Seuls
# les chaînes de 3 caractères au moins sont retenues.

items = ['10', '9', '150', '75', '3', '1540']

l = [int(s) for s in items if len(s) >= 3]

print(l)
```

```
[150, 1540]
```



Compréhension de listes

Exercice (2/3)



Ecrire un script qui construit une liste contenant les 5 premiers entiers au carré.



Compréhension de listes

Correction exercice 2/3



```
# Construit une liste contenant les 5 premiers entiers au
# carré.
nums = [0, 1, 2, 3, 4]
result = [x**2 for x in nums]

print(result)
```

```
[0, 1, 4, 9, 16]
```



Compréhension de listes

Exercice (3/3)



Ecrire un script qui comporte une liste nommée words comportant des mots commençant par le préfixe de votre choix (par exemple « cat »).

A partir de cette liste, construire une seconde liste comportant les mots de la premières liste, mis en majuscules, uniquement si ces derniers commencent par le préfixe en question.



Compréhension de listes

Correction exercice 3/3



```
# La liste initiale comportant des mots.
words = ['cat', 'dog', 'category', 'caterpillar', 'mouse',
'hedgehog', 'computer']

# Sélection des mots commençant par « cat » et mise en
# majuscule de ces derniers.
result = [w.upper() for w in words if w.startswith('cat')]

print(result)
```

```
['CAT', 'CATEGORY', 'CATERPILLAR']
```



Où en sommes-nous ?

- Données simples/composites
- Chaînes de caractères
- Tuples
- Listes
- **Dictionnaires**
- Ensembles
- Instruction for
- Comparaison for / while
- Listes complexes



Dictionnaires

Le type dict

- Le type dict est une collection non ordonnée d'éléments hétérogènes modifiables

- Syntaxe : {cle_1 : element_1, cle_2 : element_2, …, cle_n : element_n}
  - Les clés ne sont pas forcément du même type
  - Les éléments ne sont pas forcément du même type (hétérogènes)
  - Dictionnaire vide : {}
  - Dictionnaire à un seul élément : {cle : element}
  - Les éléments d'un dictionnaire peuvent être modifiés

Dictionnaires

Exemple



```
>>> d1 = {}
>>> d2 = dict()
>>> d3 = {'name' : 'Paul', 'age' : 30}

>>> d3['name']
  'Paul'

>>> d3['age']
  30
```

Opérations sur les dictionnaires

- Supposons d = `{'name':'Paul', 'age':30}` et `d2 = {}`

- Ajouter une valeur : d[cle] = valeur
- Récupérer une valeur : d.get('name')
- Supprimer une entrée : del d['name']
- Récupérer les clés : d.keys()
- Récupérer les valeurs : d.values()
- Récupérer les clés et les valeurs : d.items()
- Créer une copie indépendante : d.copy()
- Fusionner des dictionnaires : d.update(d2)

Opérations sur les dictionnaires

Exemple



```
>>> d = {
    'marty': {'first_name':'Marty',  'last_name':'Mc Fly',
'age':18},
    'doc'  : {'first_name':'Emmett', 'last_name':'Brown',
'age':50},
    'biff' : {'first_name':'Biff',   'last_name':'Tannen',
'age':18}
    }

>>> # Ajouter une valeur au dictionnaire d
>>> d['georges']={'first_name':'Georges', 'last_name':'Mc Fly',
'age':18}

>>> # Récupérer une valeur du dictionnaire d
>>> d.get('marty')
  {'first_name':'Marty', 'last_name':'Mc Fly', 'age':18}
>>> d.get('marty').get('age')
  18
```

Opérations sur les dictionnaires

Exemple



```
>>> # Supprimer une entrée du dictionnaire d
>>> del d['marty']
>>> d
  {'doc'    : {'first_name':'Emmett',
'last_name':'Brown',  'age':50},
   'biff'   : {'first_name':'Biff',
'last_name':'Tannen', 'age':18},
   'georges': {'first_name':'Georges', 'last_name':'Mc
Fly', 'age':18}}

>>> # Récupérer les clés du dictionnaire d
>>> d.keys()
  dict_keys(['doc', 'biff', 'georges'])
```

Opérations sur les dictionnaires

Exemple



```
>>> # Récupérer les valeurs d'un dictionnaire
>>> d.get('doc').values()
  dict_values(['Emmett', 'Brown', 50])

>>> # Récupérer les clés et les valeurs d'un dictionnaire
>>> d.get('doc').items()
  dict_items([('first_name','Emmett'),('last_name','Brown'),('age',50)])

>>> # Créer une copie indépendante du dictionnaire d
>>> d2 = d.copy()
>>> d2
  {'doc': {'first_name': 'Emmett', 'last_name': 'Brown', 'age': 50},
   'biff': {'first_name': 'Biff', 'last_name': 'Tannen', 'age': 18},
   'georges': {'first_name': 'Georges', 'last_name': 'Mc Fly', 'age': 18}}
```

Opérations sur les dictionnaires

Exemple



```
>>> # Fusionner deux dictionnaires
>>> d1 = {'first_name' : 'Paul'}
>>> d2 = {'last_name' : 'McCartney'}
>>> d1.update(d2)
>>> d1
  {'first_name': 'Paul', 'last_name': 'McCartney'}
```

Où en sommes-nous ?

- Données simples/composites
- Chaînes de caractères
- Tuples
- Listes
- Dictionnaires
- **Ensembles**
- Instruction for
- Comparaison for / while
- Listes complexes



Ensembles

Le type set

- Le type set est une collection non ordonnée d'éléments hétérogènes et distincts modifiables

- Syntaxe : {element_1, element_2, …, element_n}
  - Les éléments ne sont pas forcément du même type (hétérogènes)
  - Ensemble vide : {}
  - Ensemble à un seul élément : {element}
  - Les éléments d'un ensemble peuvent être modifiés
  - Les indices des éléments sont récupérables par la fonction enumerate

Opérations sur les ensembles

Supposons un ensemble s :

- Tester l'appartenance d'un élément : element in s
- Ajouter un élément : s.add(element)
- Taille d'un ensemble : len(s)
- Supprimer un élément : s.remove(element)
- Tranformer une liste l en ensemble : set(l)
- Itération : enumerate(s)
- Compréhension d'ensemble possible (cf. compréhension de liste)

Opérations sur les ensembles

Supposons deux ensembles a et b

Opérateurs ensemblistes :

- AND : a & b → éléments dans a ET dans b
- OR : a | b → éléments dans a OU dans b
- NAND : a - b → éléments dans a ET PAS dans b
  b - a → éléments dans b ET PAS dans a
- XOR : a ^ b → éléments dans a OU dans b
  MAIS PAS DANS LES DEUX

Opérations sur les ensembles

Exemple



```
>>> movie_actors = {'Marty', 'Doc', 'Biff'}

>>> # Tester l'appartenance d'un élément
>>> 'Marty' in movie_actors
  True
>>> 'Clara' in movie_actors
  False

>>> # Ajouter un élément
>>> movie_actors.add('Clara')
>>> movie_actors
  {'Clara', 'Marty', 'Doc', 'Biff'}
```

Opérations sur les ensembles

Exemple



```
>>> movie_actors
  {'Clara', 'Marty', 'Doc', 'Biff'}

>>> # Taille d'un ensemble
>>> len(movie_actors)
  4

>>> # Supprimer un élément
>>> movie_actors.remove('Doc')
>>> len(movie_actors)
  3
```

Opérations sur les ensembles

Exemple



```
>>> l = [1, 2, 3, 1, 2, 5, 2, 4, 5, 5, 6]
>>> print(l)
  [1, 2, 3, 1, 2, 5, 2, 4, 5, 5, 6]
>>> print(len(l))
  11

>>> # Tranformer une liste l en ensemble
>>> y = set(l)
>>> print(y)
  {1, 2, 3, 4, 5, 6}
>>> print(len(y))
  6
```

Opérations sur les ensembles

Exemple



```
>>> movie_actors = {'Marty', 'Doc', 'Biff'}

>>> # Enumérer les éléments d'un ensemble
>>> for idx, elem in enumerate(movie_actors) :
>>>   print(f"{idx+1}:{elem}")

>>>
  1:Doc
  2:Biff
  3:Marty
```

Opérations sur les ensembles

Exemple



```
>>> a = {1, 2, 3,    5,   7, 11}
>>> b = {1, 2, 3, 4, 5, 6}

>>> # Opérations ensemblistes
>>> print(a & b)
  {1, 2, 3, 5}
>>> print(a | b)
  {1, 2, 3, 4, 5, 6, 7, 11}
>>> print(a - b)
  {11, 7}
>>> print(b - a)
  {4, 6}
>>> print(a ^ b)
  {4, 6, 7, 11}
```

Où en sommes-nous ?

- Données simples/composites
- Chaînes de caractères
- Tuples
- Listes
- Dictionnaires
- Ensembles
- **Instruction for**
- Comparaison for / while
- Listes complexes



Instruction for

Parcours de chaînes de caractères

Exemple



```
s = 'Python'

for char in s :
  print(char)

>>>
  P
  y
  t
  h
  o
  n
```

Instruction for

Parcours de tules

Exemple



```
t = ('Python', [1, 2], 3.14)

for elem in t :
  print(elem)

>>>
  Python
  [1, 2]
  3.14
```

Instruction for

Parcours de listes

Exemple



```
l = ['Python', (1, 2), 3.14]

for elem in l :
  print(elem)

>>>
  Python
  (1, 2)
  3.14
```

Instruction for

Parcours d'un intervalle d'entiers

Exemple 1



```
for i in range(5) :
  print(i)

>>>
  0
  1
  2
  3
  4
```

Exemple 2



```
for i in range(-7, 8, 3) :
  print(i)

>>>
  -7
  -4
  -1
  2
  5
```

Où en sommes-nous ?

- Données simples/composites
- Chaînes de caractères
- Tuples
- Listes
- Dictionnaires
- Ensembles
- Instruction for
- **Comparaison for / while**
- Listes complexes



Comparaison for / while

- L'instruction for permet de parcourir uniquement des ensembles finis (dont la taille est connue avant l'exécution de la boucle)
- L'instruction while permet de créer une répétition d'instructions selon une condition quelconque

Exemple : boucle for



```
somme = 0
n = int(input(
    'Entrez un nombre : '))
for i in range(n) :
  somme = somme + i
print('Somme :', somme)

>>>
  Entrer un nombre : 100
  Somme : 4950
```

Exemple : boucle while



```
somme = 0
while somme < 100 :
  n = int(input(
    'Entrez un nombre : '))
  somme = somme + n
print('Somme :', somme)
>>>
  Entrez un nombre : 10
  Entrez un nombre : 50
  Entrez un nombre : 80
  140
```

Où en sommes-nous ?

- Données simples/composites
- Chaînes de caractères
- Tuples
- Listes
- Dictionnaires
- Ensembles
- Instruction for
- Comparaison for / while
- **Listes complexes**



Listes complexes

L'instruction for peut être utilisée avec plusieurs variables lorsque la structure à parcourir s'y prête

Exemple : parcours classique

```
l = [(0.1, 0), (1, 2), (3.14, 6.12)]

for t in l :
  x = t[0]
  y = t[1]
  print(x, y)

>>>
  0.1 0
  1 2
  3.14 6.12
```

Exemple : parcours avancé

```
l = [(0.1, 0), (1, 2), (3.14, 6.12)]

for x, y in l :
  print(x, y)

>>>
  0.1 0
  1 2
  3.14 6.12
```

Listes complexes

Exercice



On considère une liste prédéfinie l_points contenant des tuples de 2 flottants représentant des points du plan.

Ecrire un script qui demande à l'utilisateur de saisir trois flottants x_p, y_p et r et affiche tous les points de la liste se trouvant à l'intérieur du cercle de centre (x_p, y_p) et de rayon r.



Listes complexes

Correction exercice

```
# La liste l_points est à définir
l_points = [(…), (…), …, (…)]

x_p = float(input('x : '))
y_p = float(input('y : '))
r   = float(input('r : '))

for x, y in l_points :
    dX = x_p - x
    dY = y_p - y
    dist = (dX**2 + dY**2)**(0.5)
    if dist <= r :
        print(x, y)
```

